

CHAPTER 1: INTRODUCTION

1.1 Background

Heavy compound lifting demands strict form. When an athlete performs a squat, bench press, or deadlift, any deviation from safe biomechanical paths increases the risk of joint shear, muscle tears, or spinal injury. In competitive powerlifting, meeting specific mechanical rules is also the difference between a successful lift and a disqualification. The International Powerlifting Federation (IPF) enforces clear visual criteria for each movement. In the squat, the hip joint crease must pass below the top of the knee joint. The bench press requires the barbell to touch the chest, pause, and be pushed to a full lockout with both elbows extended. In the deadlift, the lifter must pull the bar from the floor and stand fully upright with locked hips, locked knees, and shoulders pulled back, ensuring the bar never travels downward during the ascent.

Traditionally, lifters check their form using mirrors or by recording videos to review after their set. However, both methods have serious drawbacks. Looking at a mirror on the side during a heavy squat or deadlift rotates the cervical spine under heavy load, which can cause severe neck strains. Post-set video reviews are helpful, but they only provide feedback after the exercise is already finished.

Recent developments in mobile hardware and on-device machine learning (Edge AI) offer a way to solve this. Modern smartphones can run pose estimation models locally to track joint landmarks in real time. The **Biomech-Coach** mobile application uses these capabilities to bridge the gap between subjective visual checks and real-time biomechanical feedback. By processing camera frames locally, the app maps key skeletal landmarks at 30 frames per second. The system calculates joint angles dynamically using trigonometric functions and counts valid repetitions. This gives lifters immediate feedback based directly on official competitive lifting standards, operating entirely on-device without any cloud requirements.

1.2 Problem Statement

Athletes training alone face several key challenges that current digital tools do not solve:

First, it is incredibly difficult to self-assess lifting depth and posture during a lift. A lifter cannot visually verify if their hip crease is deep enough when they are looking forward. Looking sideways at a mirror under a heavy barbell load is dangerous because it rotates the spine. While recording videos for later review helps, it only shows mistakes after the lift is over. If a lifter's back rounds on the first rep of a heavy deadlift set, post-workout review cannot prevent a lower back injury that happens during the lift.

Second, existing computer-vision tools rely on cloud processing. Sending high-framerate video to remote servers introduces a network delay of 500 milliseconds or more. This lag makes real-time, mid-rep voice coaching impossible. By the time the cloud server detects a rounded back and sends a warning cue, the lift is already finished or the injury has occurred. Additionally, powerlifters frequently train in basement gyms or metal warehouses where internet and cellular connectivity are poor or non-existent.

Third, traditional workout logs are highly subjective. Athletes often write down reps that were shallow or had soft lockouts as successful volume. This subjective logging pollutes training history, making it difficult to track actual progressive overload. Lifters need a system that acts as an objective filter, logging only repetitions that meet official competition standards.

1.3 Objective

The goal of this project is to develop **Biomech-Coach**, an offline mobile app that tracks compound lift biomechanics, gives real-time voice cues, and logs valid repetitions automatically. The specific objectives are:

1. To build an on-device pose estimation module using Google ML Kit to track 33 skeletal joint landmarks from a real-time camera feed.
2. To create a kinematic engine and state machine using Dart mathematical functions (like atan2) to validate Squat, Bench Press, and Deadlift form based on IPF competition rules.
3. To implement an offline coaching module that plays real-time Text-to-Speech (TTS) voice alerts for form errors and logs completed sets to a local Hive database.

1.4 Scope

This project covers the development of a cross-platform mobile application using the Flutter framework, target-compiled for Android and iOS. The app uses the device's camera to run Google ML Kit Pose Detection locally. All kinematic evaluations are performed on-device in real-time, requiring no internet connection. The system is designed to track single-person lifting attempts from a side-view camera angle.

1.4.1 Modules to be developed

The application is divided into the four main technical modules described in Table 1.1.

Module	Explanation
Real-Time Pose Tracking	Captures camera frames and runs Google ML Kit on-device to extract 33 coordinate points. Renders a live visual skeleton over the user's video feed.
Kinematic Angle Engine	Uses vector mathematics and the atan2 coordinate differences in Dart to compute real-time knee, hip, elbow, and torso angles.
Deterministic State-Machine	Tracks lift stages (idle, descending, atDepth, ascending, lockout) and triggers immediate TTS warnings (e.g., "Back rounding!", "Deeper!") for form errors.
Local Journal Database	Utilizes Hive to store workout history, filtering out invalid reps so the logged training volume only contains clean lifts.

Table 1.1: Modules to be Developed and Explanation

1.4.2 Target User

- **Competitive Powerlifters:** Athletes preparing for meets who need objective validation that their training squats hit competition depth and their deadlifts lock out fully.
- **Self-Taught Lifters:** Gym-goers performing heavy compound lifts who want real-time form checks to prevent injuries without paying for an in-person trainer.
- **Strength Coaches:** Coaches looking for objective, quantitative metrics on their athletes' joint angles, lift speeds, and postural changes.

1.5 Project Significance

This project moves biomechanical analysis from expensive labs or high-latency cloud servers straight to a user's phone. This makes high-quality feedback accessible to anyone with a smartphone.

From a safety standpoint, performing lifts with poor posture under load is a leading cause of back injuries. The immediate audio feedback loop acts as an automated safety guide, telling the athlete to correct their form or drop the weight before they hurt themselves.

From a training progress standpoint, the objective logbook ensures that athletes do not log bad repetitions. This keeps training histories clean, helping lifters make accurate training decisions based on actual quality volume rather than subjective guesses.

1.6 Expected Outcomes

The final deliverable is a functional mobile app called **Biomech-Coach** that provides:

- On-device skeletal landmark tracking running at 30 FPS with low CPU usage.
- Real-time calculation of knee, hip, elbow, and torso lean angles.
- Millisecond-responsive audio cues (like "Touch chest!" or "Lock hips!") during execution.
- A local database that logs workout history and calculates average form score percentages and specific fault frequencies.

1.7 Report Organization

The report is organized into six chapters:

1.7.1 Chapter 1: Introduction

Lays out the project background, problem statements, objectives, technical scope, and project significance.

1.7.2 Chapter 2: Literature Review and Project Methodology

Compares existing visual lift-tracking systems, reviews pose estimation models, and describes the project development methodology and requirements.

1.7.3 Chapter 3: Requirement Analysis

Details the user workflow, defines data requirements (including the data dictionary), lists functional/non-functional requirements, and justifies the software stack.

1.7.4 Chapter 4: System Design

Covers system architecture, state-machine diagrams, database schemas, and interface designs.

1.7.5 Chapter 5: Result and Discussion

Presents testing results, joint tracking accuracy comparisons, processing frame rates, and user feedback evaluations.

1.7.6 Chapter 6: Conclusion

Summarizes the final application outcomes, analyzes limitations, and suggests future improvements.

1.8 Summary

This chapter introduced the **Biomech-Coach** application as a real-time, edge-AI solution to the limitations of subjective manual logging and high-latency cloud feedback. The next chapter will review existing technologies and outline the specific methodology used to develop the application.

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY

2.1 Introduction

This chapter covers the technical background and methodology used to build **Biomech-Coach**. The review begins with the core domains: powerlifting biomechanics, on-device pose estimation, and mobile development. Next, five existing lift-tracking products are analyzed to identify what features they are missing. Different pose tracking techniques (like Google ML Kit, YOLOv8-pose, and OpenPose) are then compared, explaining why a rule-based state machine is more reliable than an ML classifier for validating lifts. Finally, the project methodology, development requirements, and milestones are laid out.

2.2 Facts and Findings

2.2.1 Domain

The project sits at the intersection of three main fields: weightlifting kinematics, edge computer vision, and mobile software engineering.

In weightlifting kinematics, compound movements like the Squat, Bench Press, and Deadlift require precise joint coordination. Dynamic form tracking requires measuring joint flexion angles and coordinate movements. In a squat, the system tracks knee flexion, hip flexion, and torso lean. For a rep to be valid under competition rules, the hip crease must drop lower than the top of the patella. In a bench press, the key metric is elbow flexion during the descent, followed by the pause on the chest and the extension phase to lockout. In the deadlift, the movement starts with a hip hinge and knee bend, ending with complete hip and knee extension at lockout. Incorrect posture, like excessive back rounding (lumbar flexion) during a deadlift or knees caving in (valgus collapse) during a squat, dramatically increases the risk of joint injuries and ligament tears.

Edge AI refers to running machine learning models locally on mobile hardware instead of remote servers. Traditional pose estimation engines require powerful desktop graphics cards. However, mobile-optimized networks (like MobileNet) allow modern phone processors to perform inference locally. Running these models on-device eliminates network

transmission times and guarantees user privacy because video frames are processed in local memory and are never saved or sent over the internet.

2.2.2 Existing Systems

Five existing fitness and lift-tracking applications were reviewed to understand their approaches and limitations.

Iron Path tracks the barbell path by identifying high-contrast plates. While it gives a clear visual line of the barbell path after the lift, it does not track the lifter's skeletal joints or calculate joint angles. It also requires the user to manually select the plate center on their screen before tracking, which is annoying during a heavy training session, and it does not give live voice cues.

Metric VBT is a mobile app that uses camera tracking to calculate barbell velocity and power. It is excellent for velocity-based training, but it is iOS-only and requires a paid subscription. It also does not track joint angles, check competition depth, or identify posture errors.

Kaia Health is a digital health platform that uses computer vision to guide rehabilitation exercises. It relies on cloud processing to analyze posture. While this works for slow physical therapy exercises, the network delay makes it useless for compound lifting feedback, where warnings must be triggered in milliseconds to prevent injuries.

MyLift is a mobile app used in research to calculate bar speed and 1RM. It requires the user to manually select the start and end frames of the lift on a video timeline. This manual approach is highly accurate for research but is too slow and tedious for everyday workouts.

FormCheck AI is a cloud-based web tool where users upload videos of their sets for joint analysis. It generates detailed post-workout reports, but because it relies on cloud uploads, it cannot provide live, mid-rep voice warnings.

No.	System / App	Developer / Source	Approach	Strength	Limitation
1	Iron Path	Science for Sport (2020)	Plate-tracking from video (Post-lift)	Shows horizontal barbell drift clearly.	Does not track joint angles; requires manual setup; post-lift only.
2	Metric VBT	Metric (2023)	Camera bar tracking (Real-time)	Accurate barbell velocity and power calculations.	Expensive; iOS only; does not analyze joint angles or depth.
3	Kaia Health	Kaia Health GmbH (2022)	Cloud-based pose estimation	High-quality guides for slow rehab exercises.	Cloud processing creates too much lag; no powerlifting rules.
4	MyLift	Balsalobre-Fernández (2018)	Manual frame-by-frame marking	Very high accuracy for research studies.	Completely manual; slow and tedious to use between sets.
5	FormCheck AI	FormCheck (2024)	Cloud web API video analysis	Detailed post-workout joint angle reports.	Cannot provide real-time audio cues during execution.

Table 2.1: Summary of Reviewed Existing Systems

2.2.3 Technique

Several pose estimation models and decision logic methods to select the best stack for our application, as shown in Table 2.2.

No.	Technique	Category	Key Characteristic	Applicability to This Project
1	Google ML Kit Pose	Pose Estimation	Tracks 33 joints, optimized for mobile CPUs.	Selected: Runs locally at 30 FPS on mid-range phones; operates completely offline.
2	YOLOv8-pose	Pose Estimation	Multi-person keypoint detection.	Not selected: Too heavy; causes phone overheating and frame drops on CPU.
3	OpenPose	Pose Estimation	Bottom-up multi-person keypoint extraction.	Not selected: Huge model size; requires desktop GPU for real-time tracking.
4	Trigonometric State Machine	Decision Logic	Evaluates joint angles using the atan2 function.	Selected: 100% predictable; easily maps to quantitative rules; no training data needed.
5	ML Rep Classifier (LSTM)	Decision Logic	Classifies reps as "good/bad" using a neural network.	Not selected: Black-box model; sensitive to camera angles; prone to false checks.

Table 2.2: Summary of Reviewed Techniques

I choose Google ML Kit Pose because it is optimized to run locally on mobile CPUs. It has a latency of under 30 milliseconds on a mid-range Android phone, avoiding the lag and battery drain of heavier models like YOLOv8-pose.

For lift validation, a rule-based state machine using trigonometric calculations rather than a machine learning classifier. An ML classifier is a black-box model that requires thousands of labeled training videos and can easily fail if a user wears baggy clothes or trains in a different room. In contrast, our math engine calculates joint angles from 2D coordinates using the atan2 function:

$$\theta = |\text{atan2}(y_c - y_b, x_c - x_b) - \text{atan2}(y_a - y_b, x_a - x_b)| \times \frac{180}{\pi}$$

This calculates the exact joint angle θ at joint vertex b (such as the knee) using coordinates a (hip) and c (ankle). These calculations are fed to a state machine that tracks the lift's progress through states: idle, descending, atDepth, ascending, lockout, and complete. This method guarantees predictable results and aligns directly with the official rule books.

2.3 Project Methodology

The development of **Biomech-Coach** follows an adapted CRISP-DM methodology. This framework provides an iterative, structured workflow that fits edge-AI software development, as shown in the diagram below:

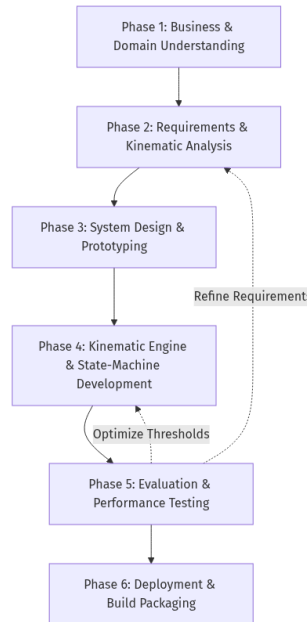


Figure 2.3: Iterative System Development Life Cycle (SDLC).

2.3.1 Phase 1: Business and Domain Understanding

The official rules of powerlifting federations (like the IPF) were analyzed to define target threshold angles for squats, benches, and deadlifts. User requirements regarding the preferred feedback types (such as live voice cues versus post-workout charts) were also gathered.

2.3.2 Phase 2: Requirements and Kinematic Analysis

The physical rules were mapped to specific joint coordinate groups (e.g. hip, knee, and ankle for squats). The trigonometric formulas were defined, and local database tables were designed to store workout logs without network requirements.

2.3.3 Phase 3: System Design and Prototyping

The application architecture was designed using the Model-View-Controller (MVC) pattern to separate camera capture from the biomechanical state-machine. Wireframe layouts were created for the camera tracking view, workout summaries, and historical logs.

2.3.4 Phase 4: Kinematic Engine and State-Machine Development

The main modules were built by integrating the Flutter camera preview with the Google ML Kit library. The math calculations in ``biomechanicsengine.dart`` and *the movement rules in statemachine.dart* were implemented. The Text-to-Speech library (`tts_coach.dart``) was also integrated to play voice alerts when the state machine detects a fault.

2.3.5 Phase 5: Evaluation and Performance Testing

The application's processing speed and tracking accuracy were tested. A test set of lifting videos was used to verify that the app counted reps and identified depth errors correctly, ensuring that frame processing did not lag on older devices.

2.3.6 Phase 6: Deployment and Build Packaging

The application was compiled into installable APK files. Tests on physical phones in offline environments were conducted to ensure the app performed camera detection, local database writes, and audio playback without crashing.

2.4 Project Requirements

2.4.1 Software Requirement

- **Development OS:** Windows 10/11 or macOS.
- **Framework:** Flutter SDK (version 3.0 or above).
- **Programming Language:** Dart.
- **Libraries:** `googlemlkitposedetection` for tracking coordinates, `hiveflutter` for local key-value storage, and `flutter_tts` for voice coaching.
- **IDE:** VS Code or Android Studio.

2.4.2 Hardware Requirement

- **Development PC:** Minimum Intel i5/Ryzen 5 processor, 16 GB RAM, and 20 GB free disk space.
- **Android Test Device:** Android 8.0 (API Level 26) or above, minimum 4 GB RAM, Snapdragon 665 or equivalent.
- **iOS Test Device:** iOS 13.0 or above.
- **Camera:** Built-in mobile camera supporting at least 720p resolution at 30 FPS.
- **Speaker:** Phone speaker or headphones for hearing voice cues.

2.4.3 Other Requirement

- **Official IPF Rule Book:** Used to calibrate the target angle constants for depth and lockouts.
- **Phone Stand / Tripod:** Required to set up the phone camera at hip height (0.5–1.0m), placed perpendicular to the lifter's side (sagittal view) at a distance of 1.5–2.5 meters.

- **Lifting Area Lighting:** A well-lit space to prevent tracking errors or joint dropouts by the pose estimation model.

2.5 Project Schedule and Milestones

The project is divided across two semesters (PSM 1 and PSM 2) to ensure structured development. Table 2.3 lists the schedule and milestones.

No.	Activity	CRISP-DM Phase	Semester	Milestone / Deliverable
1	Problem Analysis & Domain Review	Business Understanding	PSM 1	Finalized objectives, problems, and report outline.
2	Existing System Evaluation	Business Understanding	PSM 1	Comparison tables of apps and techniques.
3	Kinematic Formula Modeling	Requirements Analysis	PSM 1	Mathematical definitions for joint tracking.
4	UI & Architecture Design	Design & Prototyping	PSM 1	Wireframes and system flow diagrams.
5	PSM 1 Report Submission	Documentation	PSM 1	Submitted report draft (Chapters 1, 2, and 3).
6	Camera and ML Integration	Development	PSM 2	Working prototype capturing camera frames.
7	Math Engine Coding	Development	PSM 2	Completed joint angle calculations in code.
8	State Machine Development	Development	PSM 2	Integrated Squat, Bench, and Deadlift rules.
9	Database & Audio Cue Setup	Development	PSM 2	Configured Hive storage and Text-to-Speech alerts.
10	Usability &	Evaluation	PSM 2	Test results on

	Accuracy Testing			frame rate and tracking accuracy.
11	Final Release Build	Deployment	PSM 2	Compiled APK and installer files.
12	Report Submission & Viva	Documentation	PSM 2	Final report submission and thesis presentation.

Table 2.3: Project Schedule and Milestones

2.6 Summary

This chapter reviewed the background literature and methodology for the **Biomech-Coach** app. By analyzing existing products, a clear gap was identified: the lack of real-time, on-device form checks aligned with official powerlifting rules. Google ML Kit Pose and a math-based state machine were selected as the key technologies. The development workflow follows an adapted CRISP-DM model, supported by specific hardware and software tools. The next chapter will analyze the system requirements, covering problem flows, data dictionary, and functional specifications.

CHAPTER 3: ANALYSIS

3.1 Introduction

This chapter details the system analysis for the **Biomech-Coach** app. The chapter begins by mapping out the user workflow, comparing how athletes traditionally check their lifting form with the automated, on-device method. The database requirements are then listed, including a detailed data dictionary for the joint landmarks and database models. Finally, the functional and non-functional requirements are established, and the software engineering choices (Flutter, ML Kit, and Hive) are explained.

3.2 Problem Analysis

To understand why **Biomech-Coach** is needed, one must look at how athletes currently analyze their lifting form.

Traditionally, checking and logging form is a slow, subjective, and manual process. The typical workflow is:

1. **Setup:** The lifter props their phone against a water bottle or a gym bag on the floor.
2. **Lifting:** The lifter performs their set (e.g. 5 reps of squats) without any live feedback.
3. **Assessment:** After the set, the lifter rests, opens their photo gallery, and reviews the video. They manually drag the video timeline back and forth, guessing whether they hit squat depth or if their back rounded.

4. **Logging:** The lifter manually writes down the reps in a paper logbook or notes app, often counting reps that were too shallow because of confirmation bias.

This traditional process is shown in the workflow diagram below:

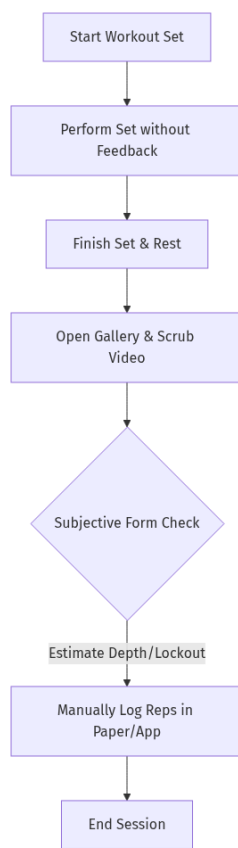


Figure 3.2.4.1: Process flow diagram of the traditional workout logging workflow

This manual workflow has three main issues:

- **Delayed Feedback:** The lifter only discovers form errors after the set is finished. If their lower back rounds during a heavy deadlift, post-set review does nothing to prevent an immediate back injury.
- **Inaccurate Training Logs:** Manual journals do not filter out bad reps. Shallow squats and soft lockouts are recorded as successful volume, creating a false record of progressive overload.

- **Parallax and Perspective Errors:** Eyeballing depth from raw video is highly subjective and depends entirely on the camera angle, often leading to incorrect self-assessment.

Biomech-Coach automates this entire pipeline on-device in real-time. The user sets up the phone camera, starts the tracking session, and performs their set. The app processes the camera frames locally, tracks skeletal coordinates, checks rules using the state machine, plays immediate voice warnings for mistakes, and saves only the valid repetitions. This automated workflow is shown below:

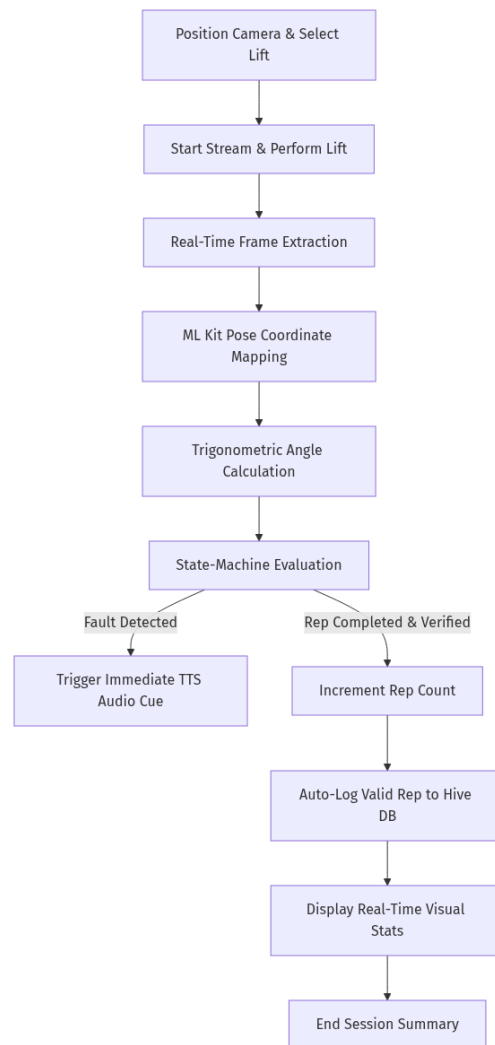


Figure 3.2.4.2: Proposed automated process flow diagram of the Biomech Coach application

3.3 Requirement Analysis

3.3.1 Data Requirements

The application processes live camera frames, transforms coordinates into mathematical joint angles, and writes session logs to local storage. The data variables are defined in the Data Dictionary in Table 3.1.

Field Name	Data Type	Source	Location	Description
landmark_coordinates	Map<PoseLandmarkType, PoseLandmark>	Google ML Kit Pose Detector	Runtime Memory	The set of 3D spatial coordinates (x, y, z) and visibility confidence values (0.0 to 1.0) extracted for the 33 tracked body joints.
joint_angle	double	BiomechanicsEngine	Runtime Memory	Calculated angle in degrees (0.0 to 180.0) for a joint vertex (e.g. knee joint angle formed by the hip, knee, and ankle points).
rep_state	enum (RepState)	StateMachine	Runtime Memory	The current active phase of the lift, tracking state transitions (idle, descending, atDepth, ascending, lockout, complete).
rep_result	RepResult Object	StateMachine	Runtime Memory	The validation summary of a completed

				rep, containing: isValid (boolean), formScore (double), minDepthAngle (double), lockoutAngle (double), and faultNotes (List<String>)
session_record	Hive Box (LiftSession)	DatabaseService	Local Hive DB	Persistent database entry storing the session details: date, lift type (Squat, Bench, Deadlift), total reps, valid reps, invalid reps, and average form score.

Table 3.1: Data Dictionary

3.3.2 Functional Requirements

The functional requirements define the core actions that the application must perform, divided into nine main requirements (FR1 to FR9).

- **FR1 - Live Camera Stream:** The app must capture and process live camera frames at a minimum of 30 frames per second.
- **FR2 - Skeletal Overlay:** The app must render a visual skeleton connect-line matching the tracked keypoints over the camera view in real-time.
- **FR3 - Angle Calculation:** The app must calculate joint flexion and torso angles locally using the atan2 trigonometric function.
- **FR4 - Lift Selection:** The app must allow the user to select their active lift type (Squat, Bench Press, or Deadlift) to load the correct rules and constants.

- **FR5 - State-Machine Tracking:** The app must run a state machine that transitions through lifting stages based on joint angle thresholds.
- **FR6 - Real-Time Voice Cues:** The app must trigger Text-to-Speech (TTS) alerts immediately (latency < 100ms) when a posture error is identified (e.g., "Back rounding!", "Deeper!").
- **FR7 - Repetition Validation:** The app must mark a completed repetition as valid only if zero posture faults were detected and all depth/lockout thresholds were met.
- **FR8 - Local Database Log:** The app must save workout logs and set summaries to local storage using Hive upon user command.
- **FR9 - History Trend Charts:** The app must load saved workouts and display progress graphs showing volume and average form scores over time.

3.3.3 Non-Functional Requirements

The non-functional requirements define the speed, reliability, and compatibility constraints of the application.

Performance

- **Inference Latency:** The frame extraction and angle calculations must execute in ≤ 33 milliseconds per frame on a Snapdragon 665 Android CPU to prevent camera lagging.
- **Voice Cue Latency:** The delay between a state machine fault detection and the start of the TTS audio alert must be ≤ 100 milliseconds.
- **Database Save Latency:** Local storage writes must complete in ≤ 10 milliseconds.

Reliability

- **Offline Independence:** All features (tracking, calculations, voice cues, and database storage) must work without cellular data or Wi-Fi.
- **Fault Tolerance:** The app must handle temporary camera occlusions (such as a plate briefly blocking a joint) without crashing, holding the state machine in the last valid state.

Usability

- **Three-Tap Workout Start:** The user must be able to start tracking a workout from the home screen in three taps or fewer.

- **Audio Voice Clarity:** Voice alerts must be clearly audible in typical gym environments with background music, using native system volume controls.

Compatibility

- **Android OS Compatibility:** The app must compile and run on devices running Android 8.0 (API Level 26) or higher.
- **iOS OS Compatibility:** The app must compile and run on devices running iOS 13.0 or higher.

Data Storage

- **Local Storage Consumption:** The local Hive database must not use more than 50 MB of storage space, even with multiple years of daily logs.
- **Installation File Size:** The compiled application bundle (APK/IPA) must be under 120 MB.

3.3.4 Other Requirements

- **ML Kit Pose Support:** The test phone must support Google Play Services (Android) or CoreML (iOS) for model acceleration.
- **Camera Placement Setup:** The user must position the phone camera at hip height (approx. 0.75m), aligned perpendicular to the lifter's side (sagittal view) at a distance of 1.5–2.5 meters.

Software Justifications

- **Flutter and Dart:** Flutter is selected because its rendering engines display 2D joint lines at a smooth 60 FPS. Dart is chosen because it allows frame processing to run on a separate background thread (isolate) so the main UI never lags or freezes.
- **Google ML Kit Pose Detection:** ML Kit was chosen over building a custom model parser because it handles coordinate normalization and landmark visibility checks automatically, reducing code complexity.
- **Hive Local Database:** Hive is chosen over SQLite because it is a lightweight, pure-Dart database that saves Dart objects directly without SQL query translation. This makes database writes almost instant and prevents lag during workouts.

3.4 Summary

This chapter analyzed the requirements for **Biomech-Coach**. By examining the traditional manual visual check workflow, the need for an automated, offline on-device processing engine. The data dictionary, functional modules, and performance limits (like the 33ms processing limit) set the specifications for the design. The next chapter will cover the system design, including database layouts, class definitions, and state transition flowcharts.